# ICT171 Cloud Server Project — Documentation

**Student number:** 35984197
**Name:** Timeo Absalon-Lheritier
**GitHub repository:** https://github.com/Murdoch16/ICT171_serveur_project
**Video explainer:** https://youtu.be/udb_BOD3_6U

## Server access details

| Item | Value |
|---|---|
| Global IP address | 209.38.25.177 |
| DNS entry (domain) | substanceinfo.xyz (and www.substanceinfo.xyz) |
| Live site | https://substanceinfo.xyz |
| Live script output | https://substanceinfo.xyz/status.html |

## 1. Project overview

This project is a cloud-hosted informational website about substance addiction (nicotine, cannabis, alcohol, methamphetamine). The goal is to provide clear and accessible health information.

The server is deployed using **Infrastructure as a Service (IaaS)**: a Linux (Ubuntu) virtual machine on DigitalOcean, configured manually over SSH. The web server (nginx), the DNS link, and the SSL/TLS certificate were all set up by hand, not from a pre-bundled image.

**Stack**

- Cloud provider: DigitalOcean (Droplet, IaaS)
- OS: Ubuntu Server 24.04 LTS
- Web server: nginx
- Domain registrar / DNS: Namecheap
- TLS: Let's Encrypt certificate issued and installed via Certbot
- Site: hand-written static HTML/CSS (multi-page)

## 2. Provision the cloud server (DigitalOcean)

1. Create a DigitalOcean account and a new Droplet.
2. Choose Ubuntu 24.04 LTS, a basic plan, and a region close to the intended audience.
3. Add an SSH key during creation.
4. Note the Droplet's public IPv4 address — this is the value used in the DNS records below.

Connect to the server over SSH:

```
ssh root@209.38.25.177
```

## 3. Initial server setup

Update the package index and installed packages:

```
apt update && apt upgrade -y
```

## 4. Install and configure nginx

Install nginx:

```
apt install nginx -y
```

The site files live in a dedicated document root:

```
mkdir -p /var/www/stayaware
```

The active server block is defined in `/etc/nginx/sites-enabled/default`. The relevant directive points the site root at the project folder:

```
server {
    root /var/www/stayaware;
    index index.html;
    server_name substanceinfo.xyz www.substanceinfo.xyz;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

After any change to the configuration, validate it and reload:

```
nginx -t                # check syntax
systemctl reload nginx  # apply
```

## 5. Deploy the website

The website is a set of hand-written HTML/CSS files (`index.html`, `nicotine.html`, `cannabis.html`, `alcohol.html`, `methamphetamine.html`, `style.css`, and an images folder). They are uploaded into the document root:

```
scp -r ./site/* root@209.38.25.177:/var/www/stayaware/
```

The site is then reachable directly by IP for testing.

## 6. Link the domain (DNS at Namecheap)

The domain `substanceinfo.xyz` was registered with Namecheap. In the Namecheap dashboard, under **Domain List → Manage → Advanced DNS**, two A records point the domain at the Droplet:

| Type | Host | Value | TTL |
|---|---|---|---|
| A Record | @ | 209.38.25.177 | Automatic |
| A Record | www | 209.38.25.177 | Automatic |

Verify the DNS resolves to the correct IP:

```
nslookup substanceinfo.xyz
```

## 7. SSL/TLS (Let's Encrypt via Certbot)

HTTPS was enabled by manually issuing a free Let's Encrypt certificate with Certbot:

```
apt install certbot python3-certbot-nginx -y
certbot --nginx -d substanceinfo.xyz -d www.substanceinfo.xyz
```

Certbot obtains the certificate, edits the nginx configuration to listen on port 443, and adds an HTTP→HTTPS redirect. Automatic renewal is handled by a systemd timer; it can be tested with:

```
certbot renew --dry-run
```

The site is now served over HTTPS at https://substanceinfo.xyz.

## 8. Script — automated server status page

A custom Bash script, `server_status.sh`, collects key health metrics from the server (uptime, disk, memory, nginx status, SSL certificate expiry) and writes them to a styled HTML page in the web root. Because the page is served by nginx, the script's output can be independently verified online at:

> https://substanceinfo.xyz/status.html

This is the student's own work (not a lab exercise): it combines several Linux administration commands and generates a public, human-readable dashboard.

Run it once:

```
sudo ./server_status.sh
```

Keep it updated automatically (every 30 minutes) via cron:

```
crontab -e
# add this line:
*/30 * * * * /root/server_status.sh
```

The full, commented source is in `server_status.sh` in this repository.

**What it does, step by step:**

1. Reads the timestamp, hostname, kernel, and uptime.
2. Reads disk usage of the root filesystem and memory usage.
3. Checks whether the nginx service is active.
4. Reads the SSL certificate's expiry date directly from the Let's Encrypt certificate file using `openssl`.
5. Writes all of this into a styled `status.html` page in the web root.

## 9. References


- DigitalOcean documentation — Droplets and initial server setup.
- nginx documentation — server blocks and `try_files`.
- Let's Encrypt / Certbot documentation — nginx plugin.
- Namecheap knowledge base — managing A records.


*(Replace with the exact URLs of any guides you actually followed, in a consistent referencing style — IEEE or APA.)*

---